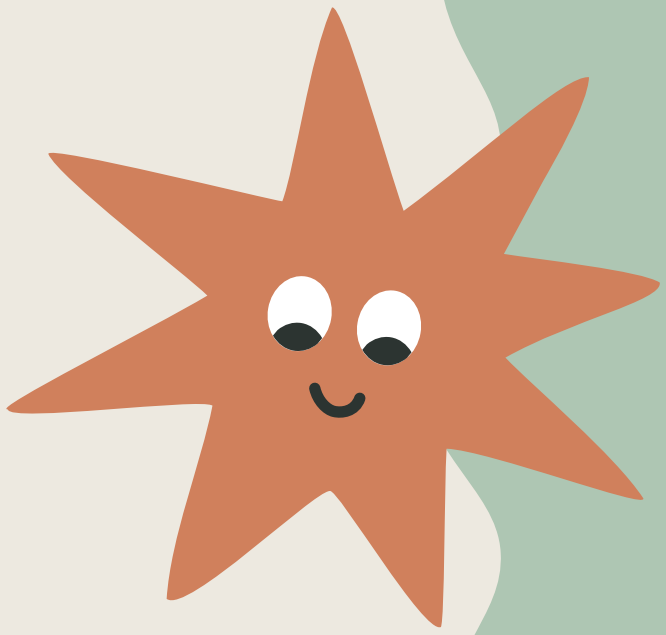
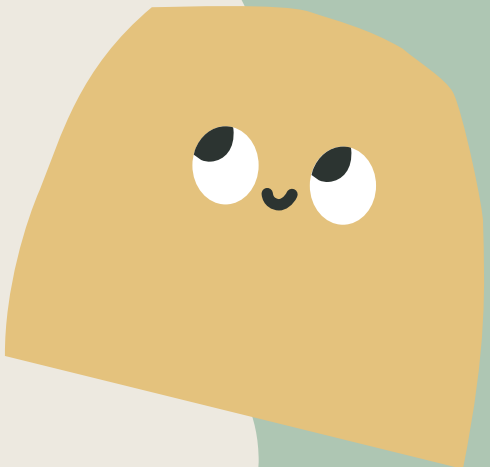
Implementation of Iterative Deepening (IDS) in Python

Lecture 15





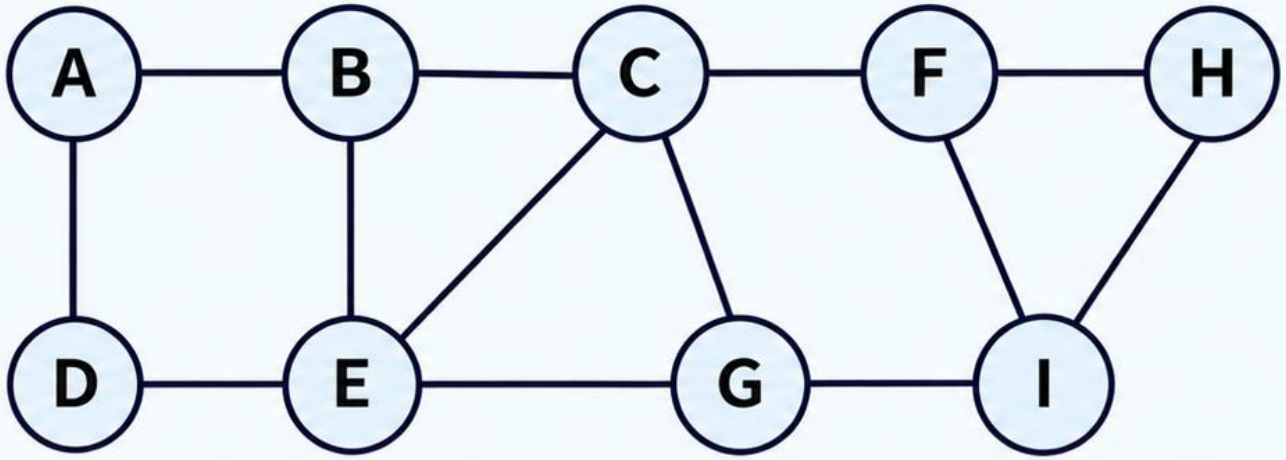
Learning Objectives

➢By the end of this lecture, students should be able to:

- Represent a search tree using a Python adjacency list.
- Implement Depth-Limited Search (DLS).
- Implement Iterative Deepening Search using repeated DLS calls.
- Understand how the depth limit changes during execution.
- Display the traversal order for each iteration.
- Distinguish between traversal order and the solution path.
- Understand the role of the parent dictionary in reconstructing the path.

Prepared By: Ehtisham Rasheed

Graph Representation



Python Representation (Adjacency List)

```python
# Graph represented as an adjacency list
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'C': ['B', 'E', 'F', 'G'],
    'D': ['A', 'E'],
    'E': ['B', 'C', 'D', 'G'],
    'F': ['C', 'H', 'I'],
    'G': ['C', 'E', 'I'],
    'H': ['F', 'I'],
    'I': ['F', 'G']
}

# Display the graph
for node, neighbors in graph.items():
    print(f"{node}: {neighbors}")
```

Understanding the Required Variables in IDS

Iterative Deepening Search (IDS) runs Depth-Limited Search (DLS) repeatedly with increasing depth limit. Each iteration starts from the initial node, so we use the following variables to control the search and keep track of the process.

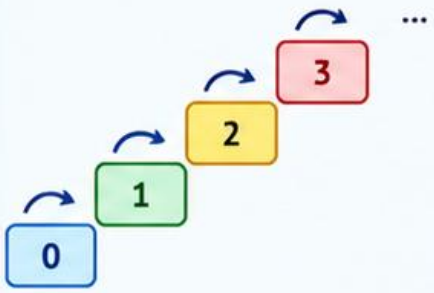
1 depth_limit

The depth_limit sets the **maximum depth allowed** in the current iteration. IDS starts with 0 and increases it by 1 in each new iteration (0, 1, 2, 3, ...).

Initialization

```
depth_limit = 0
```

Example (Iterations)



We run DLS with limit 0, then 1, then 2, and so on until the goal is found.

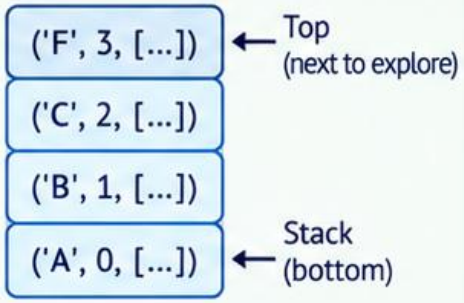
2 stack

The stack stores **nodes that are discovered but mot yet processed** during a single DLS iteration. We use a stack (LIFO) just like in DFS.

Initialization

```
stack = [(start, 0, [start}]]
```

Example (During Search)



Each entry contains:
(node, current_depth, current_path)

3 visited

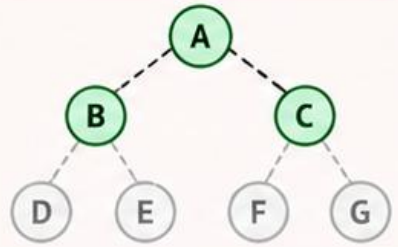
The visited set keeps track of nodes that have been discovered in the **current iteration** to avoid exploring the same node multiple times. (For graph search, we can either use visited or check against the current path.)

Initialization

```
vistek = {start}
```

Example (After some steps)

```
visited = {'A', 'B', 'C'}
```



These nodes have already been discovered in the current iteration.

4 path

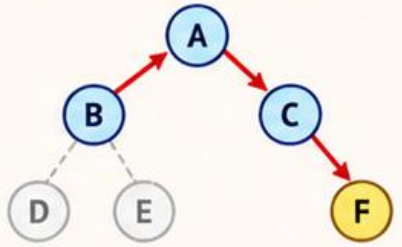
The path list stores the **route from the start node to the current node**. It helps to reconstruct the solution path when the goal is found and also prevents cycles (we do not add a node already in the current path).

Initialization

```
path = [start}
```

Example (Current Path)

```
path = ['A', 'B', 'C', 'F']
```



Stores the current route from start to the node being explored.

5 traversal

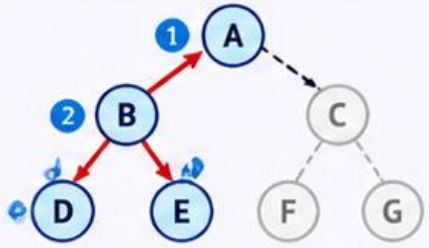
The traversal list records the **order in which nodes are processed** (popped from the stack) in the **current iteration**. It is reset in each new depth limit iteration.

Initialization

```
traversal = []
```

Example (Traversal Order)

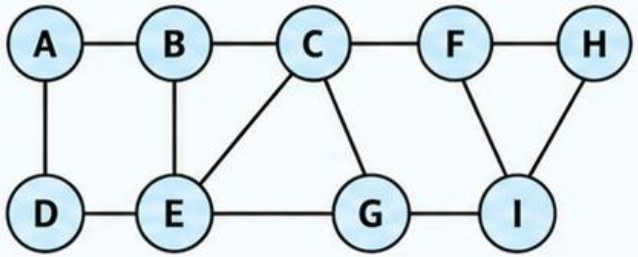
```
traversal = ['A', 'B', 'D', 'E']
```



Records the sequence of nodes processed in the current iteration.

Step One — Implement Depth-Limited Search (DLS)

1 Graph (Same as Lecture 13)



Adjacency List Representation

```
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'C': ['B', 'E', 'F', 'G'],
    'D': ['A', 'E'],
    'E': ['B', 'C', 'D', 'G'],
    'F': ['C', 'H', 'I'],
    'G': ['C', 'E', 'I'],
    'H': ['F', 'I'],
    'I': ['F', 'G', 'H']
}
```

2 Depth-Limited Search (DLS) Function

```
def depth_limited_search(graph, start, goal, depth_limit):
    # Stack entries: (node, current_depth, current_path)
    stack = [(start, 0, [start])]
    traversal = []

    while stack:
        current, depth, path = stack.pop()
        traversal.append(current)
        if current == goal:
            return traversal, path

        # Do not expand nodes at the depth limit
        if depth == depth_limit:
            continue

        # Explore neighbors in reverse order
        for neighbor in reversed(graph[current]):
            # Avoid cycles in the current path
            if neighbor not in path:
                new_path = path + [neighbor]
                stack.append((neighbor, depth + 1, new_path))
    return traversal, None
```

3 What the Function Does

1. Starts with the starting node, depth 0 and path [start].
2. Uses a stack (LIFO) to perform depth-first exploration.
3. Adds each popped node to the traversal list.
4. If the node is the goal, returns the traversal and path.
5. Does not expand neighbors when the current depth reaches the depth limit.
6. Pushes neighbors (in reverse order) if they are not already in the current path.

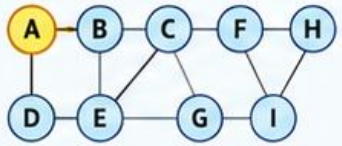
4 Return Values

traversal	List of nodes processed in this DLS iteration (in the order they were popped from the stack).
path	Solution path from start to goal if the goal is found. Otherwise, None.

5 Dry Run Example (Depth Limit = 2, Start = A, Goal = F)

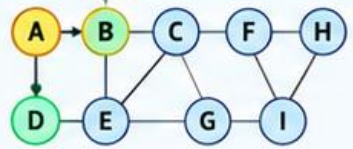
1 Start

Stack: [(A, 0, [A])]
Traversal: []
Path: -



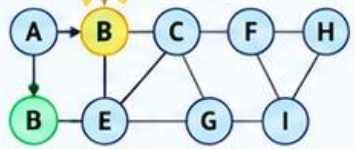
2 Pop A (depth 0)

Stack: [(D, 1, [A, D]), (B, 1, [A, B])]
Traversal: ['A']
Path: -



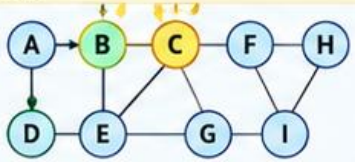
3 Pop B (depth 1)

Stack: [(D, 1, [A, D]), (E, 2, [A, B, E]), (C, 2, [A, B, C])]
Traversal: ['A', 'B']
Path: -



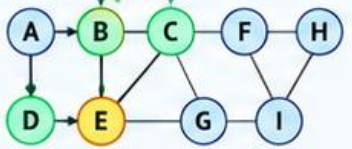
4 Pop C (depth 2)

Stack: [(D, 1, [A, D]), (E, 2, [A, B, E]), (G, 3, [A, B, C, G]), (F, 3, [A, B, C, F])]
Traversal: ['A', 'B', 'C']
Path: -



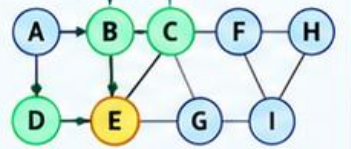
5 Pop E (depth 2)

Stack: [(D, 1, [A, D]), (G, 3, [A, B, C, G]), (F, 3, [A, B, C, F])]
Traversal: ['A', 'B', 'C', 'E']
Path: -



6 Stop (limit reached)

Next pop will be D (depth 1). Nodes at depth 2 are not expanded further, so F is not reached in this iteration.

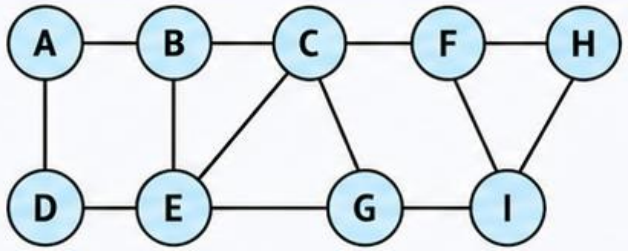


Step Two — Implement Iterative Deepening Search (IDS)

1 Main Idea

1. Start with depth limit = 0.
2. Run Depth-Limited Search (DLS) with this limit.
3. If the goal is not found, increase the depth limit by 1.
4. Repeat the process until the goal is found.

2 Graph (Same as Lecture 13)



3 IDS Function (Using DLS)

```
def ids(graph, start, goal, max_depth=20):
    all_traversals = []

    for depth_limit in range(max_depth + 1):
        print(f"\nDepth limit: {depth_limit}")

        # Run Depth-Limited Search (DLS) with current limit
        traversal, path = depth_limited_search(
            graph, start, goal, depth_limit
        )

        all_traversals.append(traversal)
        print("Traversal:"; traversal)

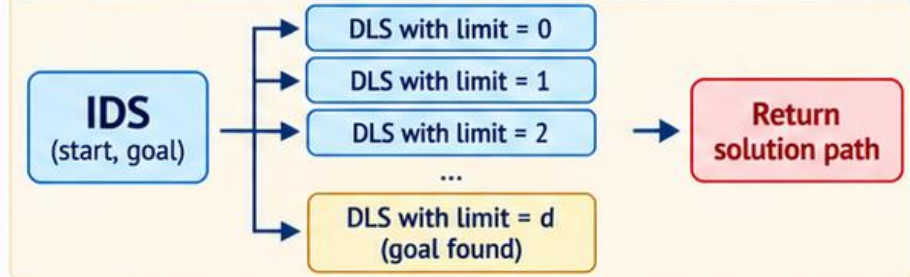
        if path is not None:
            print("Goal found!")
            print("Solution path:"; path)
            return all_traversals, path

    print("Goal not found within the maximum depth.")
    return all_traversals, None
```

4 How IDS Uses DLS

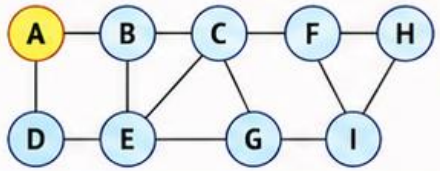
1. Calls the DLS function with a depth limit.
2. Starts from the same start node in each iteration.
3. Increases the depth limit by 1 if the goal is not found.
4. Stores the traversal of each iteration (optional).
5. Stops and returns the solution path when the goal is found.
6. Uses a maximum depth to avoid infinite search (optional, for practical implementation).

5 Function Call Flow



6 Iterative Deepening Search on the Graph (Start = A, Goal = I)

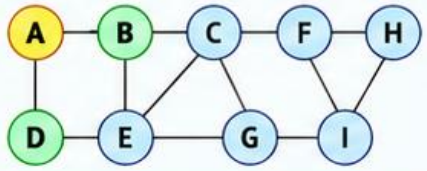
Iteration 1: Depth Limit = 0



Traversal: [A]

Goal found? **No**

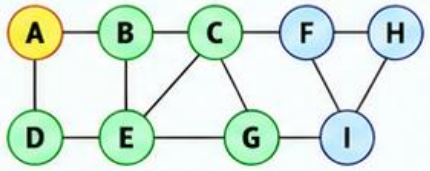
Iteration 2: Depth Limit = 1



Traversal: [A, B, D]

Goal found? **No**

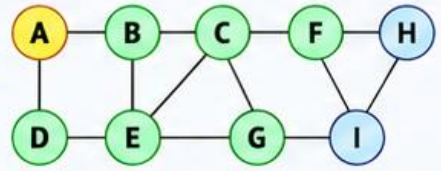
Iteration 3: Depth Limit = 2



Traversal: [A, B, C, E, D, G]

Goal found? **No**

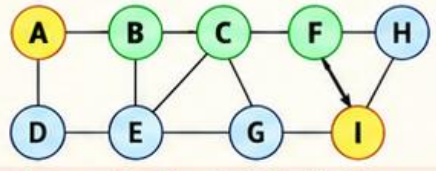
Iteration 4: Depth Limit = 3



Traversal: [A, B, C, F, E, D, G]

Goal found? **No**

Iteration 5: Depth Limit = 4



Traversal: [A, B, C, F, H, I]

Goal found? Yes

Solution Path: [A, B, C, F, I]